

PRÁCTICA PROFESIONALIZANTE 2

Alumnos:

- Prono Felipe.
- Riffel Facundo.
- Serovich Emilio.
- Villarroel Federico.

Denominación del Proyecto:

“AsistíAPP”

1. Delimitación del problema.

1.1. Descripción del problema:

La organización de eventos independientes en Argentina enfrenta un desafío operativo significativo y creciente. La gran mayoría de los productores y organizadores gestionan la venta de entradas de manera manual, recurriendo a canales informales como WhatsApp, pagos en efectivo o transferencias bancarias directas. Estas herramientas, no diseñadas para la gestión de eventos, generan una serie de problemáticas concretas que afectan tanto a los organizadores como a los asistentes.

En el contexto actual, la falta de herramientas centralizadas y accesibles para productores independientes —aquellos que no tienen acceso a plataformas corporativas como Ticketek o AllAccess— deriva en procesos ineficientes, inseguros y con una mala experiencia de usuario para todos los involucrados.

Las principales problemáticas identificadas son:

- **Falta de control y métricas en tiempo real:** Los organizadores no tienen visibilidad sobre el aforo disponible ni sobre cuántas entradas se vendieron en cada momento. Esto imposibilita la toma de decisiones basada en datos, la generación de reportes automáticos y una planificación eficiente del evento.
- **Vulnerabilidad frente al fraude y la duplicación:** El uso de entradas físicas, capturas de pantalla de comprobantes o PDFs descargables facilita la falsificación, la duplicación y la reventa ilícita. Esta problemática fue validada directamente a través del instrumento de relevamiento aplicado, donde los encuestados identificaron las "entradas falsas o compartidas" como uno de los principales puntos de dolor.

- **Desorganización en el control de accesos:** Sin herramientas digitales ágiles y confiables, el ingreso en la puerta de los eventos se torna caótico. La ausencia de un sistema de validación rápido genera colas, conflictos y situaciones de tensión en el acceso.
- **Fricción en la experiencia del comprador:** Desde la perspectiva del asistente, los procesos actuales presentan barreras considerables: la obligatoriedad de crear cuentas en plataformas externas, procesos de pago complejos, incertidumbre sobre la disponibilidad de entradas y desconfianza sobre la validez del acceso adquirido.
- **Ausencia de soluciones accesibles para el mercado local e independiente:** Las plataformas existentes en el mercado están orientadas a eventos masivos, aplican comisiones elevadas por venta y no contemplan las necesidades del productor independiente que organiza eventos medianos o pequeños en ciudades del interior del país.

1.2. Objetivos:

Objetivo General

Desarrollar una plataforma web (AsistíAPP) que permita a organizadores de eventos independientes gestionar la venta de entradas de forma digital, segura y centralizada, y que garantice a los compradores una experiencia de compra simple, sin fricciones y con validación en tiempo real mediante código QR de un solo uso.

Objetivos Específicos

- Permitir a los organizadores crear y publicar eventos con distintas tandas de precios, cupos y fechas.
- Implementar un sistema de validación de entradas mediante QR de un solo uso, accesible desde el navegador del celular sin necesidad de instalar una aplicación nativa.

- Facilitar la compra de entradas sin requerir registro previo del comprador, reduciendo la fricción en el proceso de adquisición.
- Proveer un dashboard en tiempo real para que el organizador visualice entradas vendidas, ingresos, aforo disponible y estadísticas del evento.
- Implementar un modelo de monetización basado en créditos (sin comisión porcentual por venta) para que los organizadores puedan publicar eventos de forma predecible y escalable.
- Integrar métodos de pago confiables y conocidos en el mercado local (MercadoPago) para generar confianza en los compradores.
- Diseñar la plataforma con un enfoque en la usabilidad y accesibilidad, orientada a usuarios con distintos niveles de experiencia tecnológica.

1.3. Población destinataria:

AsistíAPP está orientado a dos segmentos de usuarios principales, con roles y necesidades diferenciadas:

Segmento	Perfil	Necesidades Principales
Organizadores / Productores independientes	Personas o pequeñas productoras que organizan fiestas, recitales, festivales, eventos culturales o corporativos en ciudades intermedias.	Control de ventas, validación segura de accesos, métricas en tiempo real, modelo de costos predecible.
Compradores / Asistentes a eventos	Público general que asiste a eventos de diversa índole. Rango etario amplio, con predominio de adultos jóvenes entre 18 y 35 años.	Compra simple y rápida, sin registro obligatorio, confianza en la validez de la entrada, acceso mediante QR desde el celular.
Staff de control de accesos	Personal designado por el organizador para validar el ingreso en la puerta del evento.	Herramienta rápida y confiable de escaneo QR desde el navegador, sin necesidad de app instalada.

Geográficamente, el proyecto apunta en una primera instancia al mercado de Santa Fe y alrededores, con potencial de escalar a otras ciudades del interior del país donde las plataformas corporativas existentes tienen menor penetración y los productores independientes carecen de herramientas accesibles.

2. Fundamentación.

La organización de eventos independientes en Argentina enfrenta actualmente un desafío operativo significativo. La mayoría de los organizadores gestionan la venta de entradas de forma manual, recurriendo a canales como WhatsApp, pagos en efectivo o transferencias bancarias directas, utilizando herramientas que no fueron diseñadas específicamente para este fin. Esta informalidad y falta de herramientas centralizadas genera una serie de problemáticas concretas tanto para los productores como para los asistentes.

Desde la perspectiva tecnológica y de ingeniería de software, la problemática descrita representa una oportunidad concreta de aplicación de conocimientos adquiridos durante la formación: desarrollo de sistemas web full-stack, diseño de bases de datos relacionales, implementación de APIs REST, integración con pasarelas de pago, generación y validación de códigos QR, y diseño de interfaces centradas en el usuario.

La elección de este proyecto como trabajo práctico de Prácticas Profesionalizantes responde a los siguientes fundamentos:

- **Relevancia y validación del problema:** El problema no es hipotético. Fue identificado y validado mediante un instrumento de relevamiento (encuesta) aplicado a usuarios reales de distintos perfiles: compradores, organizadores y vendedores de entradas. Los resultados ratifican la existencia de los puntos de dolor descriptos y evidencian la demanda concreta de una solución como la propuesta.

- **Complejidad técnica apropiada para el nivel de formación:** El desarrollo de AsistíAPP implica la integración de múltiples tecnologías y capas de un sistema real: autenticación de usuarios con roles diferenciados, generación y validación de QR con lógica de negocio (un solo uso), integración con MercadoPago, dashboard con métricas en tiempo real y diseño de una base de datos relacional robusta. Este nivel de complejidad es consistente con el perfil de egreso del Técnico Superior en Desarrollo de Software.
- **Valor social y económico del producto:** AsistíAPP no es simplemente un ejercicio académico. Está concebido como un producto viable para el mercado local, orientado a democratizar el acceso a tecnología profesional de gestión de eventos para productores independientes que hoy no tienen alternativas accesibles.
- **Ausencia de soluciones equivalentes en el mercado local:** Las plataformas existentes (Ticketek, AllAccess, Passline) están orientadas a eventos masivos, aplican comisiones elevadas por venta (generalmente entre 10% y 15% del valor de la entrada) y presentan barreras de entrada para pequeños productores. TicketPass propone un modelo alternativo basado en créditos, sin comisión porcentual, más accesible y transparente para el segmento independiente.
- **Modelo de negocio innovador y sustentable:** El sistema de créditos de bienvenida gratuitos para probar la plataforma y la posterior recarga de créditos para publicar eventos —sin comisión por venta— es un diferenciador clave respecto a la competencia. Este modelo fue evaluado positivamente en el relevamiento, donde los organizadores expresaron preferencia por costos predecibles sobre comisiones variables.

En síntesis, AsistíAPP surge para resolver problemáticas reales y validadas, ofreciendo una plataforma web accesible que conecta a organizadores y compradores de forma simple, sin requerir la descarga de archivos PDF ni la instalación de aplicaciones. Al proveer un sistema con validación QR de un solo uso en tiempo real y eliminar la necesidad de registro para el comprador, se democratiza el acceso a tecnología profesional para la gestión de eventos, permitiendo un flujo de trabajo seguro, medible y eficiente.

3. Planificación de proyecto

3.1. Metodología de desarrollo:

Para abordar la complejidad técnica y garantizar la entrega de un producto funcional en el plazo estipulado de 6 meses, se adoptará Scrum como marco de trabajo ágil. Esta elección metodológica se fundamenta en la necesidad de gestionar requisitos cambiantes, priorizar la entrega de valor temprano (MVP) y mantener una comunicación constante dentro del equipo de desarrollo.

La implementación de Scrum se adaptará al contexto académico del proyecto mediante las siguientes pautas operativas:

- **Sprints:** Se establecerán iteraciones de duración fija de tres (3) semanas. Al finalizar cada Sprint, el equipo deberá entregar un incremento de software potencialmente desplegable e integrable.
- **Artefactos:** Se gestionará un *Product Backlog* priorizado que contendrá las Épicas e Historias de Usuario, y un *Sprint Backlog* técnico que desglosará las tareas a nivel de código y diseño.
- **Ceremonias:** *Sprint Planning:* Al inicio de cada iteración para comprometer la carga de trabajo.
 - *Daily Scrum (Adaptado):* Revisiones de estado trisemanal para sincronizar avances, resolver bloqueos y alinear el trabajo colaborativo.
 - *Sprint Review y Retrospective:* Al cierre del ciclo para presentar las funcionalidades completadas y aplicar mejora continua en los procesos del equipo.

3.2. Actividades a realizar: (*División en fases*)

El ciclo de vida del desarrollo se estructura en cuatro fases progresivas, diseñadas para asegurar la trazabilidad desde la concepción hasta el despliegue productivo:

1. Fase 1: Configuración Inicial y Desarrollo del Core Backend (Mes 1 - 2)

- Configuración de repositorios, políticas de ramificación (Git Flow) y preparación de los entornos base de desarrollo (IDE, entorno de ejecución de Java).
- Inicialización del proyecto Backend utilizando Spring Boot y gestión de dependencias mediante Maven o Gradle.
- Desarrollo de la arquitectura de la API RESTful estructurada en capas claras: Controladores (Controllers), Servicios (Services) y Repositorios (Repositories).
- Implementación de esquemas de seguridad robustos mediante Spring Security, configurando la autenticación y autorización basada en JWT (JSON Web Tokens) con restricciones según los roles del sistema.
- Modelado de entidades y mapeo objeto-relacional (ORM) a través de Hibernate / Spring Data JPA para la persistencia y gestión de datos en PostgreSQL.

2. Fase 2: Desarrollo Frontend e Integraciones Críticas (Mes 3 - 4)

- Maquetado funcional y programación de las interfaces de usuario en React (Dashboard de métricas, creador de eventos, Landing page de ventas).
- Consumo asíncrono de los endpoints expuestos por la API de Spring Boot desde el cliente web mediante Axios o Fetch API, gestionando los estados globales de la aplicación.
- Integración de Pasarela de Pagos: Implementación del SDK de MercadoPago en el Backend y Frontend, y desarrollo de los

controladores de escucha (Webhooks) en Spring Boot para procesar las notificaciones de pagos IPN en tiempo real.

- Desarrollo del motor generador de códigos QR unívocos asociados a las transacciones y programación de la lógica de lectura/validación óptica desde el navegador web, vinculada directamente a los servicios de verificación del backend.

3. Fase 3: Testing, Aseguramiento de Calidad (QA) y Refactorización (Mes 5)

- Ejecución de pruebas unitarias y de integración en el Backend utilizando los módulos de testing nativos de Spring (JUnit, Mockito) para asegurar la fiabilidad de la lógica de negocio.
- Pruebas de concurrencia: Simulación de múltiples peticiones HTTP simultáneas comprando entradas para validar el comportamiento de las transacciones de la base de datos de PostgreSQL (aislamiento y bloqueos) para evitar la sobreventa.
- Pruebas de campo y de usabilidad del escáner QR simulando condiciones reales (baja luminosidad, diferentes resoluciones de cámaras móviles).

4. Fase 4: Despliegue (Deployment) y Cierre (Mes 6)

- Configuración de la infraestructura, contenedores (si aplica) y bases de datos en la nube (Cloud Hosting).
- Generación del archivo ejecutable (JAR/WAR) de la aplicación Spring Boot, optimización del *build* de producción de React y despliegue en servidores productivos.
- Redacción de la documentación técnica final, manuales de usuario/despliegue y preparación de la defensa académica del sistema.

3.3. Estimación de esfuerzo:

La estimación del proyecto se realiza considerando la restricción temporal de 6 meses (aproximadamente 24 semanas netas de trabajo) enfocados netamente en la construcción y entrega del producto, con la capacidad operativa de un equipo de 4 desarrolladores.

- **Capacidad del Equipo (Velocity):** Se asume una dedicación promedio por integrante de 10 horas semanales aplicadas directamente a la programación y pruebas del proyecto.
- **Esfuerzo Semanal Global:** 40 horas/hombre.
- **Esfuerzo Total Estimado para Construcción:** ~960 horas/hombre a lo largo del ciclo de vida restante.

Estimación de esfuerzo por tareas (Fase de construcción):

- Desarrollo Backend (Spring Boot, Arquitectura y Seguridad): **~384 hs** (16 tareas estimadas — 24 hs/tarea promedio)
- Desarrollo Frontend (React, Estado Global y Maquetación): **~288 hs** (12 tareas estimadas — 24 hs/tarea promedio)
- Integraciones Críticas (MercadoPago, Algoritmos QR y JPA): **~144 hs** (6 tareas estimadas — 24 hs/tarea promedio)
- Testing, Aseguramiento de Calidad y Despliegue: **~144 hs** (6 tareas estimadas — 24 hs/tarea promedio)

3.4. Recursos humanos:

El equipo está compuesto por cuatro Técnicos Superiores en formación, operando bajo un esquema de desarrollo *Full-Stack* colaborativo. Para garantizar la eficiencia dentro del marco Scrum, se han delineado responsabilidades funcionales que los miembros asumirán a lo largo de los *sprints*:

- **Prono, Felipe:** Desarrollador Full-Stack. (Enfoque en arquitectura de base de datos y modelado de datos).
- **Riffel, Facundo:** Desarrollador Full-Stack. (Enfoque en integraciones de APIs de terceros y pasarelas de pago).
- **Serovich, Emilio:** Desarrollador Full-Stack. (Enfoque en lógica de negocio del Backend, seguridad y algoritmos de generación/validación QR).
- **Villarroel, Federico:** Desarrollador Full-Stack. (Enfoque en diseño de interfaces de usuario, experiencia interactiva en Frontend y maquetación *responsive*).

3.5. Ambiente de desarrollo, tecnologías y plataformas:

Se define un stack tecnológico basado en Spring Boot, PostgreSQL y React. La elección de estas herramientas prioriza el rendimiento, la seguridad de las transacciones financieras y una experiencia de usuario fluida, eliminando la necesidad de desarrollar aplicaciones móviles nativas. Conjunto de tecnologías y herramientas a implementar:

Capa / Área	Tecnología / Herramienta	Notas
Frontend	React.js, Tailwind o Bootstrap	Arquitectura de componentes, interfaz rápida y diseño <i>responsive</i> (Mobile First).
Backend	Java, Spring Boot	API RESTful escalable, tipado fuerte y manejo seguro de peticiones concurrentes.
Base de Datos	PostgreSQL	Motor relacional robusto. Propiedades ACID para asegurar transacciones y evitar sobreventa.
Seguridad y ORM	Spring Security, Spring Data JPA	Control de accesos apátrida (JWT) y abstracción de la base de datos (Hibernate).
Integraciones	SDK de MercadoPago, html5-qrcode	Procesamiento de pagos automático y escáner óptico web sin necesidad de app móvil.
Herramientas	Git/GitHub, Figma, Jira, Postman	Control de versiones, diseño UI/UX, seguimiento de Sprints y pruebas de <i>endpoints</i> .

3.6. Listado de casos de uso: (28 CU - 8 módulos)

Links a la Especificación de Casos de Uso y al Diagrama de Casos de Uso:

- [Especificación CU.](#)
- [Diagrama CU Autenticación.](#)
- [Diagrama CU Sistema.](#)

Módulo: Autenticación

1. CU-001 – *Iniciar sesión*
2. CU-002 – *Registrar organizador*
3. CU-003 – *Recuperar contraseña*
4. CU-004 – *Cerrar sesión*

Módulo: Gestión de Staff

5. CU-005 – *Crear Staff QR*
6. CU-006 – *Crear Staff Vendedor*

Módulo: Gestión de Eventos

7. CU-007 – *Crear evento*
8. CU-008 – *Modificar evento*
9. CU-009 – *Configurar tandas de precios y cupos*
10. CU-010 – *Publicar evento*
11. CU-011 – *Cancelar evento*

Módulo: Métricas y Créditos

12. CU-012 – *Ver métricas del evento en tiempo real*
13. CU-013 – *Consultar historial de créditos*
14. CU-014 – *Comprar créditos*

Módulo: Comprador

15. CU-015 – *Ver listado de eventos*
16. CU-016 – *Visualizar detalle del evento*
17. CU-017 – *Comprar entrada*

Módulo: Staff QR

18. CU-018 – *Escanear y validar entrada QR*

Módulo: Staff Vendedor

19. CU-019 – Registrar venta manual de entrada

Módulo: Administrador

20. CU-020 – Suspende o Activa usuario del sistema

21. CU-021 – Eliminar usuario del sistema

22. CU-022 – Reasignar rol de usuario

23. CU-023 – Editar evento del sistema (Administrador)

24. CU-024 – Cancelar evento del sistema (Administrador)

25. CU-025 – Eliminar evento del sistema (Administrador)

26. CU-026 – Administrar precios de créditos

27. CU-027 – Consultar métricas globales del sistema

28. CU-028 – Administrar configuraciones del sistema

4. Diseño de requerimientos:

4.1. Mockup's de pantallas:

Diseñados bajo un enfoque *Mobile First* utilizando Figma, estos mockups ilustran el flujo de interacción y la experiencia de usuario para los distintos perfiles del sistema (Organizador, Comprador, Staff y Administrador).

[\(Enlace al documento\)](#)

Nota: Para acceder como "Staff", se pre cargaron dos cuentas ficticias las cuales se deben clicar para autocompletar los campos de usuario y contraseña e ingresar. Ingresar una vez con cada cuenta para observar tanto la vista de Staff Vendedor como de Staff Escaneador de QR's.

Tanto para "Administrador" como "Organizador", no hace falta completar los campos de log-in.

4.2. Diagrama Entidad-Relacion:

Se presenta el modelo de datos relacional orientado a PostgreSQL. Este modelo fue diseñado con propiedades ACID para garantizar la integridad de las transacciones, evitar la sobreventa de entradas y estructurar las relaciones clave del negocio. [\(Enlace al documento\)](#)

4.3. Diagrama de tablas:

Acceso al diagrama de tablas del sistema: [\(Enlace al documento\)](#)

4.4. Diagrama de arquitectura:

El siguiente diagrama detalla la arquitectura Cliente-Servidor adoptada para la plataforma, graficando la separación de capas entre el Frontend desarrollado en React y la API RESTful del Backend construida en Java con Spring Boot. [\(Enlace al documento\)](#)